Amazon Brazil Market Analysis

Strategic Report for Amazon India

Prepared by **Omkar Malgi**

Milestone 4 - Business Analysis Fellowship, Nextleap



Executive Summary

This analysis examines Amazon Brazil's customer, payment, product, order, and sales data to identify patterns that could support Amazon India's market strategy. The analysis was performed using PostgreSQL and Excel, covering payment preferences, product pricing, customer purchasing behaviour, revenue performance, seasonal trends, customer segmentation, and sales growth.

The analysis identified several important business patterns. Credit cards are the dominant payment method, accounting for 73.9% of payment transactions, followed by Boleto at 19.0%. Seasonal analysis shows that Spring generated the highest total sales at approximately BRL 4.22 million, followed closely by Summer at approximately BRL 4.12 million.

The analysis also identified significant variation in product prices across several categories and highlighted the presence of incomplete product category information, indicating opportunities to improve product data quality.

Customer analysis shows that a large number of customers have placed multiple orders, providing an opportunity to identify and target returning and loyal customers. The analysis also uses customer segmentation and average order value to identify customers who may be suitable for loyalty and premium-reward programs.

Overall, the findings demonstrate how transaction and customer data can be transformed into actionable insights for payment strategy, seasonal planning, customer retention, product management, and revenue growth.

Introduction & Scope

Amazon has achieved significant success across international markets, including Brazil, by connecting millions of customers with sellers and small and medium-sized businesses. The similarities between Brazil and India, including large and diverse consumer markets, create an opportunity to study Amazon Brazil's data and identify patterns that could be relevant to Amazon India's expansion and growth strategy.

This project analyses Amazon Brazil's operational data from a Business Analyst perspective. The objective is not only to identify numerical trends but also to translate those trends into meaningful business insights and recommendations.

Objective

The primary objective of this analysis is to understand customer behaviour, payment preferences, product pricing, revenue performance, seasonal sales patterns, and purchasing frequency using Amazon Brazil's data.

The analysis aims to identify insights that could help Amazon India:

- Improve payment and transaction strategies.
- Understand customer purchasing behaviour.

- Identify high-value and loyal customers.
- Improve product and category data quality.
- Identify high-performing product categories.
- Understand seasonal sales patterns.
- Support inventory and promotional planning.
- Identify opportunities to improve customer retention and engagement.

Scope of Analysis

The analysis covers the following business areas:

- Payment behaviour and payment preferences.
- Product pricing and price variation.
- Customer order frequency.
- Customer segmentation.
- Product category revenue.
- Seasonal and monthly sales trends.
- High-value customer identification.
- Product sales growth.
- Monthly payment-method sales performance.
- Data quality issues in product information.

Dataset & Schema Overview

The dataset consists of interconnected tables representing different parts of Amazon Brazil's e-commerce operations. These tables allow customer, order, product, seller, and payment information to be analysed together.

Table Descriptions

Table	Purpose
Customer Data	Contains customer identifiers and location-related information. It can be used to understand customer-level purchasing behaviour and customer distribution.
Orders	Contains information about individual orders, including customer identifiers, order status, purchase timestamps, approval dates, delivery dates, and estimated delivery dates.
Order Items	Contains product-level information for each order, including product ID, seller ID, price, freight value, and shipping information.
Products	Contains product-level attributes such as product category, product name length, description length, number of product photographs, weight, length, height, and width.
Sellers	Contains seller identifiers and location-related information, allowing seller-level analysis.
Payments	Contains payment transaction information including payment type, payment sequence, number of installments, and payment value.

Tables (Created Using SQL CREATE TABLE command)

```sql
-- 1. Customer Data
CREATE TABLE amazon_brazil.customer_data (
customer_id VARCHAR(50) PRIMARY KEY,
customer_unique_id VARCHAR(50) NOT NULL,
customer_zip_code_prefix VARCHAR(10) NOT NULL
);
```

```sql
-- 2. Order Items
CREATE TABLE amazon_brazil.order_items (
order_id VARCHAR(50) NOT NULL,
order_item_id INT NOT NULL,
product_id VARCHAR(50) NOT NULL,
seller_id VARCHAR(50) NOT NULL,
shipping_limit_date TIMESTAMP NOT NULL,
price NUMERIC(10,2) NOT NULL,
freight_value NUMERIC(10,2) NOT NULL,
PRIMARY KEY (order_id, order_item_id)
);
```

```sql
-- 3. Payments
CREATE TABLE amazon_brazil.payments (
order_id VARCHAR(50) NOT NULL,
payment_sequential INT NOT NULL,
payment_type VARCHAR(20) NOT NULL,
payment_installments INT NOT NULL,
payment_value NUMERIC(10,2) NOT NULL,
PRIMARY KEY (order_id, payment_sequential)
);
```

```sql
-- 4. Orders
CREATE TABLE amazon_brazil.orders (
order_id VARCHAR(50) PRIMARY KEY,
customer_id VARCHAR(50) NOT NULL,
order_status VARCHAR(20) NOT NULL,
order_purchase_timestamp TIMESTAMP NOT NULL,
order_approved_at TIMESTAMP,
order_delivered_carrier_date TIMESTAMP,
order_delivered_customer_date TIMESTAMP,
order_estimated_delivery_date TIMESTAMP NOT NULL
);
```

```sql
-- 5. Products
CREATE TABLE amazon_brazil.product (
product_id VARCHAR(50) PRIMARY KEY,
product_category_name VARCHAR(100),
product_name_lenght INT,
product_description_lenght INT,
product_photos_qty INT,
product_weight_g INT,
product_length_cm INT,
product_height_cm INT,
product_width_cm INT
);
```

```sql
-- 6. Sellers
CREATE TABLE amazon_brazil.sellers (
seller_id VARCHAR(50) PRIMARY KEY,
seller_zip_code_prefix VARCHAR(10) NOT NULL
);
```

Methodology

The analysis was conducted using PostgreSQL and Excel (for selected visualizations). The overall analytical process followed these steps:

1. Import the Amazon Brazil dataset into PostgreSQL.
2. Create the required database schema and tables.
3. Establish relationships between the relevant tables.
4. Use SQL queries to extract and aggregate the required information.
5. Analyse payment, product, customer, revenue, and sales patterns.
6. Export selected results to Excel for visualization.
7. Interpret the results from a business perspective.
8. Translate key findings into recommendations relevant to Amazon India.

The SQL analysis used several techniques, including:

- Aggregate functions such as SUM, AVG, COUNT, MIN, MAX and STDDEV.
- GROUP BY and HAVING.
- INNER JOIN.
- CASE statements.
- Subqueries.
- Common Table Expressions (CTEs).
- Window functions.
- Recursive CTEs.
- Date and time functions.
- Filtering and sorting.

ANALYSIS - I

Que 1

> **To simplify its financial reports, Amazon India needs to standardize payment values. Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.**

SQL QUERY

```sql
SELECT payment_type, ROUND(AVG(payment_value)) AS avg_payment_value
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY avg_payment_value ASC
```

OUTPUT

	payment_type *character varying (20)*	avg_payment_value *numeric*
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

> **KEY INSIGHT / RECOMMENDATION**
>
> Credit card has the highest average payment value among the listed payment types, while voucher has a much lower average. Amazon India can use these differences to design payment-specific offers and transaction strategies.

Que 2

> **To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type. Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order.**

SQL QUERY

```sql
SELECT payment_type, ROUND(COUNT(payment_type)*100/SUM(COUNT(*)) OVER(), 1) AS
percentage_total
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY percentage_total DESC
```

OUTPUT

	payment_type *character varying (20)*	percentage_total *numeric*
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

> **KEY INSIGHT / RECOMMENDATION**
>
> Credit cards account for 73.9% of payment transactions, making them the dominant payment method, followed by Boleto at 19.0%. Amazon India should prioritize a smooth credit-card checkout experience while continuing to support alternative methods.

Que 3

> **Amazon India seeks to create targeted promotions for products within specific price ranges. Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.**

SQL QUERY

```sql
SELECT o.product_id, o.price, p.product_category_name
FROM amazon_brazil.order_items o
JOIN amazon_brazil.product p
ON p.product_id = o.product_id
WHERE o.price BETWEEN 100 AND 500
AND p.product_category_name ILIKE '%smart%'
ORDER BY o.price DESC
```

OUTPUT

	product_id *character varying (50)*	price *numeric (10,2)*	product_category_name *character varying (100)*
1	1df1a2df8ad2b9d3aa49fd851e314...	439.99	smart
2	7debe59b10825e89c1cbcc8b190c...	349.99	smart
3	ca86b9fe16e12de698c955aedff0ae...	349.00	smart
4	ca86b9fe16e12de698c955aedff0ae...	349.00	smart
5	0e52955ca8143bd179b311cc454a...	335.00	smart
6	7aeaa8f3e592e380c420e8910a717...	329.90	smart
7	7aeaa8f3e592e380c420e8910a717...	329.90	smart
8	7aeaa8f3e592e380c420e8910a717...	329.90	smart
9	7aeaa8f3e592e380c420e8910a717...	329.90	smart
10	7aeaa8f3e592e380c420e8910a717...	329.90	smart

Total rows: 34 | Query complete 00:00:00.439

> **KEY INSIGHT / RECOMMENDATION**
>
> The analysis identified 34 matching product records within the specified price range containing "Smart" in the category/name field. These products could be considered for targeted promotions aimed at customers interested in smart products.

Que 4

> **To identify seasonal sales patterns, Amazon India needs to focus on the most successful months. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.**

SQL QUERY

```sql
SELECT EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
ROUND(SUM(oi.price)) AS total_sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
ON oi.order_id = o.order_id
GROUP BY month
ORDER BY total_sales DESC
LIMIT 3
```

OUTPUT

	month *numeric*	total_sales *numeric*
1	5	1502589
2	8	1428658
3	7	1393539

> **KEY INSIGHT / RECOMMENDATION**
>
> May generated the highest sales at approximately BRL 1.50M, followed by August and July. These stronger-performing months can be used as reference periods for promotional and inventory planning.

Que 5

> **Amazon India is interested in product categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.**

SQL QUERY

```sql
SELECT p.product_category_name,
MAX(o.price) - MIN(o.price) AS price_difference
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items o
ON o.product_id = p.product_id
GROUP BY p.product_category_name
HAVING MAX(o.price) - MIN(o.price) > 500
```

OUTPUT

	product_category_name *character varying (100)*	price_difference *numeric*
1	climatizacao	1588.10
2	livros_importados	730.01
3	*[null]*	3977.00
4	ferramentas_jardim	3923.65
5	dvds_blu_ray	1411.10
6	cine_foto	867.19
7	beleza_saude	3122.80
8	livros_interesse_geral	893.90
9	tablets_impressao_imagem	875.09
10	papelaria	1690.71

Total rows: 57 | Query complete 00:00:00.413

> **KEY INSIGHT / RECOMMENDATION**
>
> The analysis identified 57 categories/records with price differences above BRL 500, showing substantial pricing variation across the dataset. Amazon India should investigate the drivers of these differences, such as product type, seller, or product specifications, before making pricing decisions.

Que 6

> **To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts. Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.**

SQL QUERY

```sql
SELECT payment_type, STDDEV(payment_value) AS standard_deviation
FROM amazon_brazil.payments
GROUP BY payment_type
ORDER BY standard_deviation
```

OUTPUT

	payment_type *character varying (20)*	standard_deviation *numeric*
1	not_defined	0
2	voucher	115.519185430458
3	boleto	213.581061475527
4	credit_card	222.119310741208
5	debit_card	245.793401040647

> **KEY INSIGHT / RECOMMENDATION**
>
> Among the payment types shown, voucher has the lowest non-zero standard deviation (115.52), indicating relatively more consistent transaction values than the other commonly used methods. This can help identify payment methods associated with more predictable transaction sizes.

Que 7

> **Amazon India wants to identify products that may have incomplete name in order to fix it from their end. Retrieve the list of products where the product category name is missing or contains only a single character.**

SQL QUERY

```sql
SELECT product_id, product_category_name
FROM amazon_brazil.product
WHERE product_category_name IS NULL
OR LENGTH(product_category_name) = 1
```

OUTPUT

	product_id *[PK] character varying (50)*	product_category_name *character varying (100)*
1	a41e356c76fab66334f36de622ecbd...	*[null]*
2	d8dee61c2034d6d075997acef1870...	*[null]*
3	56139431d72cd51f19eb9f7dae4d16...	*[null]*
4	46b48281eb6d663ced748f324108c...	*[null]*
5	5fb61f482620cb672f5e586bb132ea...	*[null]*
6	e10758160da97891c2fdcbc35f0f03...	*[null]*
7	39e3b9b12cd0bf8ee681bbc1c130fe...	*[null]*
8	794de06c32a626a5692ff50e4985d3...	*[null]*
9	7af3e2da474486a3519b0cba9dea8...	*[null]*
10	629beb8e7317703dcc5f35b5463fd2...	*[null]*

Total rows: 614 | Query complete 00:00:00.238

KEY INSIGHT / RECOMMENDATION

The analysis identified 614 product records where the category is missing or contains only one character. This is a significant data-quality issue that should be addressed to improve product categorization and the reliability of category-level analysis.

ANALYSIS - II

Que 1

> **Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high). Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count.**

SQL QUERY

```sql
SELECT
(CASE
    WHEN payment_value < 200 THEN 'LOW'
    WHEN payment_value BETWEEN 200 AND 1000 THEN 'MEDIUM'
    ELSE 'HIGH'
END) AS order_value_segment,
payment_type, COUNT(payment_type) AS count
FROM amazon_brazil.payments
GROUP BY order_value_segment, payment_type
ORDER BY count DESC
```

OUTPUT

	order_value_segment *text*	payment_type *character varying (20)*	count *bigint*
1	LOW	credit_card	60548
2	LOW	boleto	16444
3	MEDIUM	credit_card	15303
4	LOW	voucher	5476
5	MEDIUM	boleto	3162
6	LOW	debit_card	1287
7	HIGH	credit_card	944
8	MEDIUM	voucher	286
9	MEDIUM	debit_card	227
10	HIGH	boleto	178

Total rows: 13 | Query complete 00:00:00.610

KEY INSIGHT / RECOMMENDATION

Credit cards are the most frequently used payment method across the low, medium, and high order-value segments shown. Amazon India should maintain strong credit-card support while tailoring payment offers to different order-value segments.

Que 2

Amazon India wants to analyse the price range and average price for each product category. Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.

SQL QUERY

```
SELECT p.product_category_name,
MIN(o.price) AS min_price,
MAX(o.price) AS max_price,
ROUND(AVG(o.price), 2) AS avg_price
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items o
ON o.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC
```

OUTPUT

	product_category_name *character varying (100)*	min_price *numeric*	max_price *numeric*	avg_price *numeric*
1	pcs	34.50	6729.00	1098.34
2	portateis_casa_forno_e_cafe	10.19	2899.00	624.29
3	eletrodomesticos_2	13.90	2350.00	476.12
4	agro_industria_e_comercio	12.99	2990.00	341.66
5	instrumentos_musicais	4.90	4399.87	281.62
6	eletroportateis	6.50	4799.00	280.78
7	portateis_cozinha_e_preparadores_de_aliment...	17.42	1099.00	264.57
8	telefonia_fixa	6.00	1790.00	225.69
9	construcao_ferramentas_seguranca	8.90	3099.90	208.99
10	relogios_presentes	8.99	3999.90	200.91

Total rows: 79 | Query complete 00:00:00.451

KEY INSIGHT / RECOMMENDATION

The results show considerable differences between minimum, maximum, and average prices across product categories. Categories with higher average prices may require different pricing, promotion, and inventory strategies from lower-value categories.

Que 3

Amazon India wants to identify the customers who have placed multiple orders over time. Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.

SQL QUERY

```
SELECT c.customer_unique_id,
COUNT(o.order_id) AS total_orders
FROM amazon_brazil.customer_data c
JOIN amazon_brazil.orders o
ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(o.order_id) > 1
```

OUTPUT

	customer_unique_id *character varying (50)*	total_orders *bigint*
1	00172711b30d52eea8b313a7f2cce...	2
2	004288347e5e88a27ded2bb237470...	2
3	004b45ec5c64187465168251cd1c9...	2
4	0058f300f57d7b93c477a131a59b36...	2
5	00a39521eb40f7012db50455bf083...	2
6	00cc12a6d8b578b8ebd21ea4e2ae8...	2
7	011575986092c30523ecb71ff10cb4...	2
8	011b4adcd54683b480c4d841250a9...	2
9	012452d40dafae4df401bced74cdb4...	2
10	012a218df8995d3ec3bb221828360...	2

Total rows: 3140 | Query complete 00:00:02.493

KEY INSIGHT / RECOMMENDATION

The query identified 3,140 customers with more than one order, confirming the presence of a meaningful returning-customer base. These customers can be targeted with retention campaigns, personalized offers, and loyalty benefits.

Que 4

Amazon India wants to categorize customers into different types ('New' - order qty. = 1; 'Returning' - order qty. 2 to 4; 'Loyal' - order qty. > 4) based on their purchase history. Use a temporary table to define these categories and join it with the customers table to update and display the customer types.

SQL QUERY

```sql
CREATE TEMP TABLE customer_types AS
SELECT c.customer_unique_id,
(CASE
    WHEN COUNT(o.order_id) = 1 THEN 'New'
    WHEN COUNT(o.order_id) BETWEEN 2 AND 4 THEN 'Returning'
    ELSE 'Loyal'
END) AS customer_type
FROM amazon_brazil.customer_data c
JOIN amazon_brazil.orders o
ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id;

SELECT
c.customer_unique_id,
ct.customer_type
FROM amazon_brazil.customer_data c
JOIN customer_types ct
ON c.customer_unique_id = ct.customer_unique_id;
```

OUTPUT

	customer_unique_id *character varying (50)*	customer_type *text*
1	0000366f3b9a7992bf8c76cfdf3221...	New
2	0000b849f77a49e4a4ce2b2a4ca5b...	New
3	0000f46a3911fa3c0805444483337...	New
4	0000f6ccb0745a6a4b88665a16c9f...	New
5	0004aac84e0df4da2b147fca70cf82...	New
6	0004bd2a26a76fe21f786e4fbd8060...	New
7	00050ab1314c0e55a6ca13cf7181f...	New
8	00053a61a98854899e70ed204dd4...	New
9	0005e1862207bf6ccc02e4228effd9...	New
10	0005ef4cd20d2893f0d9fbd94d3c0d...	New

Total rows: 98422 | Query complete 00:00:03.218

KEY INSIGHT / RECOMMENDATION

The analysis classifies customers into New, Returning, and Loyal segments based on their order frequency, creating a useful foundation for differentiated customer strategies. Amazon India can use these segments to tailor acquisition, retention, and loyalty campaigns.

Que 5

Amazon India wants to know which product categories generate the most revenue. Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.

SQL QUERY

```
SELECT p.product_category_name,
SUM(o.price) AS total_revenue
FROM amazon_brazil.product p
JOIN amazon_brazil.order_items o
ON o.product_id = p.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5
```

OUTPUT

	product_category_name *character varying (100)*	total_revenue *numeric*
1	beleza_saude	1257865.34
2	relogios_presentes	1203060.32
3	cama_mesa_banho	1032268.59
4	esporte_lazer	985881.10
5	informatica_acessorios	910605.07

KEY INSIGHT / RECOMMENDATION

Beauty & Health generated the highest revenue among the top five categories at approximately BRL 1.26M, followed by Watches & Gifts and Bed/Bath/Table. These high-performing categories could receive greater focus in promotions and inventory planning.

ANALYSIS - III

Que 1

> **The marketing team wants to compare the total sales between different seasons. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.**

SQL QUERY

```sql
SELECT season, SUM(price) AS total_sales
FROM (
    SELECT
    CASE
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (3, 4, 5) THEN
'Spring'
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (6, 7, 8) THEN
'Summer'
        WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp) IN (9, 10, 11) THEN
'Autumn'
        ELSE 'Winter'
    END AS season,
    oi.price AS price
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
) AS sales_data
GROUP BY season;
```

OUTPUT

	season *text*	total_sales *numeric*
1	Spring	4216721.54
2	Summer	4120359.62
3	Winter	2905750.03
4	Autumn	2348812.51

KEY INSIGHT / RECOMMENDATION

Spring generated the highest seasonal sales at approximately BRL 4.21M, closely followed by Summer at BRL 4.12M. Marketing and inventory planning should therefore pay particular attention to these stronger-performing seasons.

Que 2

The inventory team is interested in identifying products that have sales volumes above the overall average. Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

SQL QUERY

```
SELECT product_id,
COUNT(order_item_id) AS total_quantity_sold
FROM amazon_brazil.order_items
GROUP BY product_id
HAVING COUNT(order_item_id) > (
    SELECT AVG(total_quantity_sold)
    FROM (
        SELECT product_id,
        COUNT(order_item_id) AS total_quantity_sold
        FROM amazon_brazil.order_items
        GROUP BY product_id
    )
)
```

OUTPUT

	product_id *character varying (50)*	total_quantity_sold *bigint*
1	3d5837f86205fe83f03fb5f7e4d5b9cf	11
2	afeeea6271148ee1bb15173b8187c4	53
3	434487f82b5c35646bd8155cf19461...	4
4	e5063ce7fff1cf7cd528dc4c1e7dcba8	4
5	b25a0f93e25104798df2d1664495d1...	4
6	6639a238ead6779d6ef0b3eea56f9f86	4
7	dceb3f67aef3484498a7caa4ba50f484	6
8	3c4e3782469a0f1ac459dc6c47ebef31	4
9	ecaaaccb5eb3102553f001d62db638...	6
10	98ad26989524a790f1d29686025b6f...	4

Total rows: 6366 | Query complete 00:00:00.558

KEY INSIGHT / RECOMMENDATION

The query identified 6,366 products whose sales volume is above the calculated overall average. These products represent a broad group of higher-volume items and can be further analysed to identify the strongest products for inventory and promotional planning.

Que 3

To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018). Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.

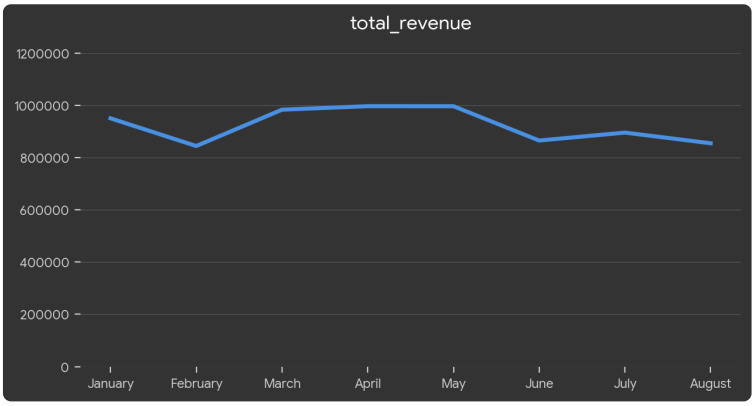
SQL QUERY

```
SELECT EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
SUM(oi.price) AS total_revenue
FROM amazon_brazil.order_items oi
JOIN amazon_brazil.orders o
ON o.order_id = oi.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
GROUP BY month
ORDER BY month
```

OUTPUT

total_revenue	Month
950030.36	January
844178.71	February
983213.44	March
996647.75	April
996517.68	May
865124.31	June
895507.22	July
854686.33	August
145.00	September

MAX	MIN
996647.75	844178.71
April	February



KEY INSIGHT / RECOMMENDATION

Among the months visible in the output, April has the highest revenue at approximately BRL 996.65K, while February is the lowest at approximately BRL 844.18K. The monthly trend can help identify stronger and weaker periods for future sales and inventory planning.

Que 4

> **A loyalty program is being designed for Amazon India. Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.**

SQL QUERY

```sql
WITH customer_orders AS(
    SELECT customer_id, COUNT(order_id) AS order_count
    FROM amazon_brazil.orders
    GROUP BY customer_id
),
customer_segments AS(
    SELECT customer_id,
    CASE
        WHEN order_count BETWEEN 1 AND 2 THEN 'Occasional'
        WHEN order_count BETWEEN 3 AND 5 THEN 'Regular'
        ELSE 'Loyal'
    END AS customer_type
    FROM customer_orders
)
SELECT customer_type, COUNT(*) AS count
FROM customer_segments
GROUP BY customer_type
```

OUTPUT

	customer_type *text*	count *bigint*
1	Occasional	98144
2	Regular	106
3	Loyal	98

> **KEY INSIGHT / RECOMMENDATION**
>
> The segmentation shows 98,144 Occasional customers, compared with 106 Regular and 98 Loyal customers. This indicates a very large opportunity to convert occasional buyers into repeat customers through targeted retention and loyalty initiatives.

Que 5

> **Amazon wants to identify high-value customers to target for an exclusive rewards program. You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.**

SQL QUERY

```sql
WITH order_values AS (
    SELECT order_id, SUM(price) AS order_value
    FROM amazon_brazil.order_items
    GROUP BY order_id
),
customer_avg AS(
    SELECT o.customer_id, AVG(ov.order_value) AS avg_order_value
    FROM amazon_brazil.orders o
    JOIN order_values ov
    ON o.order_id = ov.order_id
    GROUP BY o.customer_id
)
SELECT customer_id, avg_order_value,
RANK() OVER (ORDER BY avg_order_value DESC) AS customer_rank
FROM customer_avg
ORDER BY customer_rank
LIMIT 20
```

OUTPUT

	customer_id *character varying (50)*	avg_order_value *numeric*	customer_rank *bigint*
1	161761357756262bfa56ab541c47b...	13440.0000000000000000	1
2	ec5b2ba62e574342386871631fafd...	7160.0000000000000000	2
3	c6e2731c5b391845f6800c97401a4...	6735.0000000000000000	3
4	f48d464a0baaea338cb25f816991ab...	6729.0000000000000000	4
5	3fd6777bbce08a352fddd04e4a7cc8...	6499.0000000000000000	5
6	05455dfa7cd02f13d132aa7a6a972...	5934.6000000000000000	6
7	df55c14d1476a9a3467f131269c24...	4799.0000000000000000	7
8	24bbf5fd2f2e1b359ee7de94defc4a15	4690.0000000000000000	8
9	e0a2412720e9ea4f26c1ac985f6a73...	4599.9000000000000000	9
10	3d979689f636322c6241866346b1c...	4590.0000000000000000	10

Total rows: 20 | Query complete 00:00:02.219

KEY INSIGHT / RECOMMENDATION

The ranking identifies customers with substantially higher average order values, with the highest visible customer averaging BRL 13,440 per order. These customers are strong candidates for exclusive rewards, personalized offers, and premium customer-retention initiatives.

Que 6

Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.

SQL QUERY

```
WITH RECURSIVE monthly_sales AS (
    SELECT oi.product_id,
    DATE_TRUNC('month', o.order_purchase_timestamp)::date AS sale_month,
    SUM(oi.price) AS monthly_sales
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
    GROUP BY oi.product_id, DATE_TRUNC('month', o.order_purchase_timestamp)
),
numbered_sales AS (
    SELECT product_id, sale_month, monthly_sales,
    ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY sale_month) AS month_no
    FROM monthly_sales
),
cumulative_sales AS (
    SELECT product_id, sale_month, month_no, monthly_sales AS total_sales
    FROM numbered_sales
    WHERE month_no = 1
    UNION ALL
    SELECT n.product_id, n.sale_month, n.month_no,
    c.total_sales + n.monthly_sales AS total_sales
    FROM cumulative_sales c
    JOIN numbered_sales n
    ON c.product_id = n.product_id
    AND n.month_no = c.month_no + 1
)
SELECT product_id, sale_month, total_sales
FROM cumulative_sales
ORDER BY product_id, sale_month;
```

OUTPUT

	product_id *character varying (50)*	sale_month *date*	total_sales *numeric*
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05-01	101.65
2	00088930e925c41fd95ebfe695fd2655	2017-12-01	129.90
3	0009406fd7479715e4bef61dd91f2462	2017-12-01	229.00
4	000b8f95fcb9e0096488278317764d...	2018-08-01	117.80
5	000d9be29b5207b54e86aa1b1ac54...	2018-04-01	199.00
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12-01	52.00
7	00126f27c813603687e6ce486d909d...	2017-09-01	498.00
8	001795ec6f1b187d37335e1c470476...	2017-10-01	38.90
9	001795ec6f1b187d37335e1c470476...	2017-11-01	116.70
10	001795ec6f1b187d37335e1c470476...	2017-12-01	350.10

Total rows: 62176 | Query complete 00:00:06.063

KEY INSIGHT / RECOMMENDATION

The output contains 62,176 product-month records and shows cumulative sales tracked month-by-month for each product. This provides a useful basis for monitoring how sales build over a product's lifecycle and identifying products with stronger or weaker sales progression.

Que 7

To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018). Write a query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.

SQL QUERY

```
WITH monthly_sales AS (
    SELECT p.payment_type, DATE_TRUNC('month', o.order_purchase_timestamp)::date AS
sale_month,
    SUM(p.payment_value) AS monthly_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.payments p
    ON o.order_id = p.order_id
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
    GROUP BY p.payment_type, DATE_TRUNC('month', o.order_purchase_timestamp)
),
```

```
sales_with_previous AS (
    SELECT payment_type, sale_month, monthly_total,
    LAG(monthly_total) OVER (PARTITION BY payment_type ORDER BY sale_month) AS
previous_month
    FROM monthly_sales
)
SELECT payment_type, sale_month, monthly_total,
ROUND(((monthly_total - previous_month) / NULLIF(previous_month, 0)) * 100, 2) AS
monthly_change
FROM sales_with_previous
ORDER BY payment_type, sale_month
```

OUTPUT

	payment_type *character varying (20)*	sale_month *date*	monthly_total *numeric*	monthly_change *numeric*
1	boleto	2018-01-01	204844.66	*[null]*
2	boleto	2018-02-01	183112.72	-10.61
3	boleto	2018-03-01	191538.02	4.60
4	boleto	2018-04-01	193547.09	1.05
5	boleto	2018-05-01	195378.93	0.95
6	boleto	2018-06-01	153350.28	-21.51
7	boleto	2018-07-01	198041.24	29.14
8	boleto	2018-08-01	143805.90	-27.39
9	credit_card	2018-01-01	868880.38	*[null]*
10	credit_card	2018-02-01	778803.00	-10.37

Total rows: 36 | Query complete 00:00:00.774

KEY INSIGHT / RECOMMENDATION

The visible results show noticeable month-to-month fluctuations, particularly for Boleto, with changes ranging from +29.14% to -27.39% in the displayed months; Credit Card also declined by 10.37% in February. Amazon India should monitor these monthly changes to identify periods of declining payment-method performance and investigate the underlying causes.

Key Findings

The analysis highlights the following major findings:

- **Credit cards** are the dominant payment method, accounting for **73.9%** of payment transactions. Boleto is the second most-used method at **19.0%**.
- Seasonal and monthly sales vary significantly, with **Spring** generating the highest seasonal sales at approximately **BRL 4.21M**.
- Product pricing varies substantially across categories, with **57 categories/records** showing a price difference above **BRL 500**.
- Product data quality needs improvement, with **614 records** having missing or single-character category names.
- Returning customers represent an important opportunity, with **3,140 customers** having placed more than one order.
- **Beauty & Health** is the highest-revenue category among the top five categories, generating approximately **BRL 1.26M**.
- A very large proportion of customers are occasional buyers, creating an opportunity to convert them into regular and loyal customers.
- High-value customers can be identified using average order value, supporting targeted premium rewards and retention strategies.

Business Recommendations

1. Strengthen Credit Card Payment Experience

Prioritize a smooth and reliable credit-card checkout experience while continuing to support alternative payment methods.

2. Use Seasonal Demand for Planning

Align promotions, inventory, and marketing campaigns with high-performing seasons and months.

3. Improve Product Data Quality

Clean and standardize missing or incomplete product-category information to improve reporting and product discoverability.

4. Build Customer Retention Programs

Target occasional and returning customers with personalized offers, loyalty benefits, and repeat-purchase campaigns.

5. Develop High-Value Customer Programs

Use average order value and purchase frequency to identify customers eligible for premium rewards and personalized offers.

6. Focus on High-Performing Categories

Prioritize strong revenue-generating categories for inventory, promotions, and seller strategy.

7. Monitor Payment Sales Trends

Track monthly payment-method performance to identify unusual declines and investigate their causes.

Conclusion

This analysis demonstrates how Amazon Brazil's transaction, customer, product, and payment data can be transformed into actionable business insights. The findings highlight opportunities in payment strategy, customer retention, seasonal planning, product data quality, pricing, and revenue growth. For Amazon India, these insights can support more informed decisions around customer engagement, promotions, inventory planning, and payment strategy.

Skills & Tools Demonstrated

- **Tools:** PostgreSQL, Excel
- **SQL Skills:** Aggregate Functions, GROUP BY, HAVING, Joins, CASE Statements, Subqueries, CTEs, Window Functions, Recursive CTEs, Date and Time Functions, Filtering and Sorting.